MOCHAVI SECURITY ASSESSMENT

DUSK ERC20 Audit Report

Formal review of the verified deployment, exposed control surface, and observable supply-management properties.

PREPARED BY
Mochavi for Dusk

ASSESSMENT DATE
April 22, 2026

NETWORK
Ethereum mainnet

COMPILER
v0.4.25+commit.59dbf8f1

ADDRESS
0x940a2db1b7008b6c776d4faaca729d6d4a4aa551

Scope

Item	Value
Contract	`DuskToken`
Network	Ethereum mainnet
Address	`0x940a2db1b7008b6c776d4faaca729d6d4a4aa551`
Verified source	https://etherscan.io/token/0x940a2db1b7008b6c776d4faaca729d6d4a4aa551#code
Reviewed artifact	`DuskToken.sol` verified-source replica
Compiler	`v0.4.25+commit.59dbf8f1`
Optimizer	Disabled (`0` runs)

Methodology and Severity Classification

This assessment consisted of manual review of the verified contract source, inspection of the externally exposed token surface, and live RPC validation of present on-chain supply

state. The review focused on post-deployment control surfaces, supply mutability, transfer semantics, approval mechanics, and the presence or absence of privileged administrative functionality.

Severity ratings in this report are assigned according to the following rubric:

- **Critical**: direct and irrecoverable loss of user funds, unrestricted theft, or catastrophic supply compromise requiring no privileged assumptions beyond ordinary network conditions.
- **High**: unbacked minting, privileged-role seizure, or other defects that can materially compromise token integrity or user balances under reachable conditions.
- **Medium**: meaningful accounting or control-plane weaknesses, or architectural trust dependencies that can produce security-relevant impact but do not amount to immediate catastrophic failure.
- **Low**: narrow-scope issues, standard integration hazards, or hardening concerns with limited direct impact.
- **Informational**: descriptive observations, deployment-context notes, or non-defective implementation characteristics recorded for completeness.

Executive Summary

The Ethereum DUSK contract is a minimal fixed-supply ERC20. All `500,000,000` tokens are minted once in the constructor and assigned to the deployer, after which the contract exposes only standard ERC20 transfer and allowance flows. No owner role, external mint, blacklist, fee, pause, proxy, upgrade, or privileged post-deployment control surface was identified.

The review therefore indicates a comparatively low-risk fixed-supply token implementation. The principal residual consideration is the well-known ERC20 allowance race associated with direct `approve()` transitions. A live RPC query performed on April 22, 2026 confirmed `totalSupply() = 500000000000000000000000000` (`500,000,000 DUSK`).

Security Properties Summary

Check	Status	Notes
External minting	Pass	No externally reachable mint function detected.
Privileged owner controls	Pass	No ownership or admin role exists in the source or ABI.
Upgradeability / proxy	Pass	No proxy, delegatecall, or upgrade hooks detected.
Blacklist / pause / fee logic	Pass	No transfer taxes, pause flags, blocklists, or max-wallet logic detected.
Reentrancy surface	Pass	No external calls are made during token state transitions.
Allowance safety	Caution	Direct `approve()` preserves the conventional ERC20 allowance race condition.

Findings Summary

Severity	Count
Critical	0
High	0
Medium	0
Low	1
Informational	0

Detailed Findings

L-01: Standard ERC20 allowance race remains on direct `approve()`

Severity: LOW
Location: `DuskToken.sol:174`

The contract allows a holder to change a non-zero allowance directly to a different non-zero allowance through `approve()`. This behavior corresponds to the standard ERC20 allowance race: a spender may consume the prior allowance immediately before the new approval is mined, thereby creating a window in which both authorizations can be exercised across transaction ordering.

This behavior is not unique to DUSK, and the in-code commentary explicitly acknowledges it. The contract partially mitigates the issue by also exposing `increaseAllowance()` and `decreaseAllowance()` at `DuskToken.sol:212` and `DuskToken.sol:236`.

Impact: Integrations or users that rely on raw `approve()` updates may expose themselves to allowance front-running.
Recommendation: Prefer `increaseAllowance()` / `decreaseAllowance()`, or set allowance to `0` before assigning a new non-zero value.

Notes

- The token is fixed supply by construction: the only `_mint()` call appears in the constructor at `DuskToken.sol:322`.
- Internal burn helpers exist in inherited code, but no external burn entrypoint is exposed.
- The Solidity compiler version is consistent with the historical deployment period and is not treated in this report as a standalone adverse finding.

Conclusion

The Ethereum DUSK token presents a narrow and well-understood ERC20 attack surface with no live privileged controls after deployment. Supply is fixed, and no administrative authority remains after construction.

Sources

- Etherscan verified source: https://etherscan.io/token/0x940a2db1b7008b6c776d4faaca729d6d4a4aa551#code
- Etherscan contract page: https://etherscan.io/address/0x940a2db1b7008b6c776d4faaca729d6d4a4aa551